Urban Thermal Network Math Deep Dive

Scope and intent

This document explains, in plain language and in mathematical detail, the full modeling stack used across:

* `spectral-urbanism`
* `spectral_urbanism_boston`

It covers:

1. Data to graph modeling
2. Laplacian and spectral graph theory
3. Cheeger cut and conductance
4. Probability, percolation, and reliability
5. Combinatorics and computational hardness
6. GMRF inference
7. Optimization objective and greedy selection
8. Practical scientific interpretation and limitations
9. A fully worked toy example
10. A layperson glossary mapped to repository modules
11. An equations-only appendix

---

1) The core idea in plain language

Both repositories translate urban heat and vegetation maps into a network.

* A map cell becomes a node.
* Neighboring cells are connected by edges.
* Each edge has a weight (conductance): larger means easier cooling linkage.
* Weak links form bottlenecks.
* Spectral graph math finds those bottlenecks.
* Probability models random edge failures to test robustness.
* Optimization picks intervention locations that improve connectivity and fairness.

Think of the city as a thermal road network. If a few narrow bridges are weak, the entire city cooling flow is fragile. The Cheeger cut finds those weak bridges.

---

2) Data to graph construction

2.1 Grid and nodes

In `spectral-urbanism/core/graph.py`, each finite raster cell (LST and optional NDVI) is a node.

In `spectral_urbanism_boston/spectral_urbanism/graph/build.py`, each city grid cell (GeoDataFrame row) is a node.

2.2 Edges and neighborhood

* Raster repo: 4-neighbor or 8-neighbor connectivity (rook/queen style).
* Boston repo: polygon touch adjacency, plus optional nearest-neighbor wind proxy edges.

2.3 Edge weights (conductance)

Raster repo (`spectral-urbanism`)

A local temperature gradient is computed. Steeper gradient means a stronger thermal barrier.

Weight form:

$$w_{ij} = \exp(-\alpha g_{ij}) \cdot (1 + \beta \cdot \text{ndvi}_{ij})$$

where:

* g_{ij} is local gradient magnitude around edge (i,j)
* $\alpha > 0$ controls barrier sensitivity
* $\beta \ge 0$ controls NDVI uplift

Then cost is inverse-like:

$$\text{cost}_{ij} = \frac{\ell_{ij}}{w_{ij}}$$

with $\ell_{ij}=1$ for cardinal adjacency and $\sqrt{2}$ for diagonal.

Boston repo (`spectral_urbanism_boston`)

A configurable linear feature score is used:

$$w_{ij} \approx a_1\,\text{albedo} + a_2\,\text{ndvi} + a_3\,\text{wind} - a_4\,\text{impervious} - a_5\,\text{distance}$$

Then clipped to a small positive floor.

Interpretation:

* NDVI and albedo typically increase thermal connectivity quality.
* Imperviousness and distance reduce it.
* Wind term exists but is currently simplified in parts of implementation.

---

3) Laplacian mathematics ("Laplace everything")

3.1 Matrices

For weighted graph $G=(V,E,W)$:

* W (weighted adjacency): $W_{ij}=w_{ij}$ if edge exists, else 0
* Degree at node i: $d_i = \sum_j W_{ij}$
* Degree matrix $D = \text{diag}(d_1,\dots,d_n)$

3.2 Laplacians

Combinatorial Laplacian:

$$L = D - W$$

Normalized Laplacian:

$$L_{\text{norm}} = I - D^{-1/2} W D^{-1/2}$$

Both repos use normalized Laplacian for spectral metrics.

Why normalized? It avoids over-favoring high-degree nodes and makes comparison more stable across heterogeneous degree distributions.

3.3 Eigenvalues and eigenvectors

* Eigenvalues of L_{norm}: $0=\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$
* λ_2 is algebraic connectivity (spectral gap in this workflow).
* Eigenvector for λ_2 is Fiedler vector.

Intuition:

* Larger λ_2: more globally well-connected thermal network.
* Smaller λ_2: easier to split, more bottlenecked.

---

4) Cheeger cut, conductance, and sweep

4.1 Conductance definition

For node subset $S \subset V$:

$$\phi(S)=\frac{\text{cut}(S,V\setminus S)}{\min(\text{vol}(S),\text{vol}(V\setminus S))}$$

where:

$$\text{cut}(S,V\setminus S)=\sum_{i\in S,\,j\notin S} w_{ij}$$

$$\text{vol}(S)=\sum_{i\in S} d_i$$

Low conductance means the graph has a narrow weak bridge between two bigger regions.

4.2 Why spectral sweep

Exact minimization over all subsets is combinatorially expensive. Number of possible nontrivial cuts grows exponentially.

The practical method:

1. Compute Fiedler vector.
2. Sort nodes by Fiedler value.
3. Sweep prefix sets of this ordering.
4. Compute conductance for each prefix.
5. Choose best (minimum conductance) prefix.

This is implemented in:

* `spectral-urbanism/core/spectra.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/cheeger.py`

4.3 Cheeger boundary nodes

After best split is found, edges crossing the split are identified. Endpoints of crossing edges become bottleneck boundary nodes.

Those are treated as "corridor pinch points" for intervention focus.

---

5) Probability and percolation

5.1 Edge-failure model

Each edge is retained with probability p and removed with probability 1-p, independently.

This defines a random subgraph G_p.

5.2 Percolation scan

For each p in a grid (for example 0.1 to 1.0):

* sample many random subgraphs
* compute largest connected component fraction
* average across samples

This traces robustness phase behavior: how quickly connectivity collapses under random failures.

Implemented in:

* `spectral-urbanism/core/percolation.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/reliability.py` (scan function)

5.3 Reliability metrics

Two related reliability views exist in the repos:

* Sink-reachability reliability (raster repo): expected fraction of nodes connected to at least one cooling sink under random failures.
* All-terminal reliability (Boston repo): probability the entire graph remains connected under random failures.

Both are Monte Carlo estimators in current implementation.

---

6) Combinatorics and computational complexity

6.1 Why this is combinatorics-heavy

1. Cut search:
* Candidate subsets are exponential in node count.
2. Reliability exact computation:
* Requires summation over 2^m edge states in brute force form.
3. Intervention search:

* Candidate set size grows with number of nodes times intervention types.

6.2 Practical approximations used

* Spectral sweep instead of brute-force cut minimization.
* Monte Carlo reliability instead of exact all-state enumeration.
* Greedy marginal-gain selection instead of full combinatorial optimization.

These are standard tractability compromises for city-scale graphs.

---

7) Cooling sinks and resistance proxy

7.1 Sink inference

Sinks are inferred from cool or green cells (quantile thresholds on NDVI and/or temperature).

7.2 Super-sink shortest path

A synthetic super-sink is attached with zero cost to all sink nodes.

Edge resistance cost is inverse conductance:

$$r_{ij} \propto \frac{1}{w_{ij}}$$

Then Dijkstra shortest path from super-sink gives each node's resistance-to-cooling distance.

7.3 Access normalization

Distances are transformed to access score in [0,100], where high means easier sink access.

In Boston repo, the resistance proxy field is often represented as:

$$\text{resistance\_proxy} = 100 - \text{cooling\_access\_score}$$

---

8) Priority synthesis for bottleneck mitigation

A weighted blend is used on bottleneck boundary cells:

$$\text{priority} = 100\left(0.65\cdot \text{heat\_unit} + 0.35\cdot (1-\text{access\_unit})\right)$$

Interpretation:

* High heat increases urgency.
* Poor access to cooling sinks increases urgency.
* Restricting to Cheeger boundary cells targets structural bottlenecks rather than all hot cells.

---

9) GMRF in the Boston stack

9.1 Prior

Using graph Laplacian as smoothness prior:

$$Q = \tau L + \epsilon I$$

where:

* τ controls smoothness strength
* ϵ stabilizes precision matrix and avoids singular issues

9.2 Posterior with Gaussian observations

Observed nodes add diagonal precision:

$$Q_{\text{post}} = Q + \frac{1}{\sigma^2}I_{obs}$$

Posterior mean solve:

$$\mu = Q_{\text{post}}^{-1} b$$

This gives a graph-regularized temperature field used in objective evaluation.

---

10) Objective function and optimization

Boston objective:

$$\text{score} = \alpha\,\lambda_2 + \beta\,\text{reliability} - \gamma\,\text{equity\_exposure}$$

with equity exposure defined as weighted burden:

$$\text{equity\_exposure} = \sum_i \text{vulnerability}_i \cdot \text{temp}_i$$

Greedy algorithm at each step:

1. Try each candidate intervention.
2. Apply local edge-weight multiplier around target node.
3. Recompute score.
4. Pick best positive gain candidate.
5. Repeat until budget exhausted.

This is implemented in:

* `spectral_urbanism_boston/spectral_urbanism/opt/interventions.py`
* `spectral_urbanism_boston/spectral_urbanism/opt/objective.py`
* `spectral_urbanism_boston/spectral_urbanism/opt/greedy.py`

---

11) Fully worked toy example (requested)

This section is intentionally explicit and hand-computable.

11.1 Example A: 5-node path, uniform edge weight 1

Graph:

1 - 2 - 3 - 4 - 5

Edge weights all 1.

Degrees:

* d_1=1
* d_2=2
* d_3=2
* d_4=2
* d_5=1

Total volume:

\text{vol}(V)=1+2+2+2+1=8

Assume Fiedler ordering is monotone left-to-right (true for path).

Sweep table over prefixes:

1) S_1=\{1\}

* cut = edge (1,2) = 1
* vol(S1)=1
* vol(comp)=7
* denom=min(1,7)=1
* \phi(S_1)=1/1=1.0

2) S_2=\{1,2\}

* cut = edge (2,3) = 1
* vol(S2)=1+2=3
* vol(comp)=5
* denom=3
* \phi(S_2)=1/3=0.3333

3) S_3=\{1,2,3\}

* cut = edge (3,4) = 1
* vol(S3)=1+2+2=5
* vol(comp)=3
* denom=3
* \phi(S_3)=1/3=0.3333

4) S_4=\{1,2,3,4\}

* cut = edge (4,5) = 1
* vol(S4)=7
* vol(comp)=1
* denom=1
* \phi(S_4)=1.0

Best sweep conductance is 1/3 at S2 or S3.

Interpretation: middle split is weakest normalized separator in this simple graph.

11.2 Example B: same path but weak middle bridge

Edges:

* (1,2)=1
* (2,3)=0.2 (weak bridge)
* (3,4)=1
* (4,5)=1

Degrees:

* d_1=1
* d_2=1.2
* d_3=1.2
* d_4=2
* d_5=1

Total volume:

\text{vol}(V)=1+1.2+1.2+2+1=6.4

Sweep around weak bridge split S={1,2}:

* cut = w(2,3)=0.2
* vol(S)=1+1.2=2.2
* vol(comp)=4.2
* denom=2.2
* \phi=0.2/2.2=0.0909

This is much lower than uniform case 0.3333.

Interpretation: one weak edge creates a strong bottleneck signal. This is exactly what the Cheeger corridor is trying to detect in spatial thermal networks.

11.3 Reliability exact micro-example

For a path graph with 5 nodes and 4 edges, all-terminal connectivity requires all 4 edges survive.

If each edge survives independently with probability p:

R_{all-terminal} = p^4

At p=0.7:

R=0.7^4=0.2401

Monte Carlo reliability estimators in code should converge near 0.2401 with enough draws.

11.4 Percolation micro intuition

At low p, many edges fail, giant component fraction is small.
At high p, giant component includes most nodes.

For path-like sparse graphs, this transition is smoother than dense urban graphs.

---

12) Science interpretation for planning

12.1 What a high-priority bottleneck means

A high Cheeger priority cell means:

* it lies on a structural split boundary,
* it is locally hot,
* and/or it has weak sink access.

These are high-value candidates for corridor-style interventions that reconnect cooling pathways.

12.2 Why this is not full physics

This framework is a graph-theoretic proxy model, not a full CFD urban canopy simulation.

Strengths:

* tractable at city scale,
* auditable equations,
* robust comparative optimization under fixed assumptions.

Limits:

* edge effects are simplified multipliers,
* microclimate fluid dynamics are not explicitly solved,
* reliability uses Monte Carlo baseline estimators in current version.

---

13) Layperson glossary mapped to code (requested)

13.1 Core graph terms

* Node: one map/grid cell.
* `spectral-urbanism/core/graph.py`
* `spectral_urbanism_boston/spectral_urbanism/graph/build.py`

* Edge: neighboring cell relationship.
* same modules as above

* Weight or conductance: ease of thermal linkage.
* same modules as above

* Cost or resistance: inverse-like travel difficulty to sinks.
* `spectral-urbanism/core/pipeline.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/cooling_access.py`

13.2 Spectral terms

* Laplacian: matrix encoding graph structure.
* `spectral-urbanism/core/graph.py`
* `spectral_urbanism_boston/spectral_urbanism/graph/laplacian.py`

* lambda2: second-smallest normalized Laplacian eigenvalue.
* `spectral-urbanism/core/spectra.py`

* `spectral_urbanism_boston/spectral_urbanism/metrics/spectral.py`

* Fiedler vector: eigenvector linked to lambda2 used to rank nodes for sweep cuts.
* `spectral-urbanism/core/spectra.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/cheeger.py`

* Cheeger conductance: normalized cut quality of a set.
* same cheeger/spectra modules

13.3 Probability terms

* Bond percolation: random edge keep/remove process with keep probability p.
* `spectral-urbanism/core/percolation.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/reliability.py`

* Reliability: probability network remains connected or sink-reachable under random failures.
* `spectral-urbanism/core/reliability.py`
* `spectral_urbanism_boston/spectral_urbanism/metrics/reliability.py`

* Monte Carlo estimator: repeated random simulation to approximate expected value or probability.
* same reliability/percolation modules

13.4 Inference and optimization terms

* GMRF: Gaussian Markov Random Field graph-based spatial prior/posterior model.
* `spectral_urbanism_boston/spectral_urbanism/model/gmrf.py`

* Objective: weighted score combining connectivity, reliability, and equity.
* `spectral_urbanism_boston/spectral_urbanism/opt/objective.py`

* Greedy selection: iterative best-next intervention choice.
* `spectral_urbanism_boston/spectral_urbanism/opt/greedy.py`

---

14) Equations-only technical appendix (requested)

14.1 Graph and Laplacian

$$d_i = \sum_j w_{ij}$$

$$D = \text{diag}(d_1,\ldots,d_n)$$

$$L = D - W$$

$$L_{\text{norm}} = I - D^{-1/2}WD^{-1/2}$$

14.2 Spectral quantities

$$0=\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$$

$$\text{mixing-time upper bound} \propto \frac{\log(1/\varepsilon)}{\lambda_2}$$

14.3 Conductance and Cheeger sweep

$$\text{cut}(S,V\setminus S)=\sum_{i\in S, j\notin S} w_{ij}$$

$$\text{vol}(S)=\sum_{i\in S} d_i$$

$$\phi(S)=\frac{\text{cut}(S,V\setminus S)}{\min(\text{vol}(S),\text{vol}(V\setminus S))}$$

14.4 Sink resistance and access

$$r_{ij} = \frac{\ell_{ij}}{\max(w_{ij},\epsilon)}$$

$$d_i = \text{shortest-path distance from node } i \text{ to super-sink}$$

$$\text{access}_i = 100\cdot (1-\text{normalized}(d_i))$$

14.5 Priority blending

$$\text{priority}_i = 100\cdot\mathbf{1}_{\{i\in\text{boundary}\}}\left(0.65\,h_i + 0.35\,(1-a_i)\right)$$

where $h_i\in[0,1]$ is normalized heat and $a_i\in[0,1]$ is normalized access.

14.6 Reliability and percolation

$$R_{all}(p)=\Pr(G_p \text{ is connected})$$

$$\hat R_{all} = \frac{1}{T}\sum_{t=1}^T \mathbf{1}\{G_p^{(t)}\text{ connected}\}$$

$$\text{GCF}(p)=\mathbb{E}\left[\frac{|C_{max}(G_p)|}{|V|}\right]$$

14.7 GMRF

$$Q = \tau L + \epsilon I$$

$$Q_{post} = Q + \frac{1}{\sigma^2}I_{obs}$$

$$\mu = Q_{post}^{-1}b$$

14.8 Composite objective

$$\text{score}=\alpha\lambda_2 + \beta R - \gamma E$$

$$E=\sum_i v_i\,t_i$$

where v_i is vulnerability and t_i is temperature (or posterior mean proxy).

---

15) Practical checklist for scientific use

1. Validate edge-weight calibration per city.
2. Report confidence intervals for Monte Carlo reliability.
3. Run ablations (no spectral, no reliability, no equity).
4. Stress-test sensitivity to grid resolution and quantile thresholds.
5. Document policy constraints before operational deployment.

---

16) Final takeaway

The repositories implement a coherent graph-spectral-probabilistic urban heat framework:

* Cheeger and lambda2 capture structural thermal connectivity.
* Percolation and reliability capture failure robustness.
* Cooling sink resistance captures practical access deficits.
* GMRF and equity-aware objective support decision scoring.

Scientifically, this is a strong decision-support proxy framework and not a full physical fluid-dynamics simulator. It is most powerful for comparative planning, prioritization, and transparent tradeoff analysis under explicit assumptions.